



图解大模型 生成式AI原理与实战

Hands-On Large Language Models
Language Understanding and Generation

[沙特] 杰伊·阿拉马尔 (Jay Alammar) 著
[荷] 马尔滕·格鲁滕多斯特 (Maarten Grootendorst)
李博杰 译

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

O'Reilly Media, Inc. 授权人民邮电出版社有限公司出版

人 民 邮 电 出 版 社
北 京

图书在版编目 (CIP) 数据

图解大模型：生成式 AI 原理与实战 / (沙特) 杰伊

• 阿拉马尔 (Jay Alammar), (荷) 马尔滕 • 格鲁滕多斯特

特 (Maarten Grootendorst) 著 ; 李博杰译. -- 北京 :

人民邮电出版社, 2025. -- ISBN 978-7-115-67083-0

I . TP18

中国国家版本馆 CIP 数据核字第 2025FB3467 号

内 容 提 要

本书全程图解式讲解, 通过大量全彩插图拆解概念, 让读者真正告别学习大模型的枯燥和复杂。

全书分为三部分, 依次介绍语言模型的原理、应用及优化。第一部分“理解语言模型”(第 1~3 章), 解析语言模型的核心概念, 包括词元、嵌入及 Transformer 架构, 帮助读者建立基础认知。第二部分“使用预训练语言模型”(第 4~9 章), 介绍如何使用大模型进行文本分类、聚类、语义搜索、文本生成及多模态扩展, 提升模型的应用能力。第三部分“训练和微调语言模型”(第 10~12 章), 探讨大模型的训练与微调方法, 包括嵌入模型的构建、分类任务的优化及生成模型的微调, 以适应特定需求。

本书适合对大模型感兴趣的开发者、研究人员和行业从业者。读者无须具备深度学习基础知识, 只要会用 Python, 就可以通过本书深入理解大模型的原理并上手大模型应用开发。书中示例还可以一键在线运行, 让学习过程更轻松。

-
- ◆ 著 [沙特] 杰伊 • 阿拉马尔 (Jay Alammar)
[荷] 马尔滕 • 格鲁滕多斯特 (Maarten Grootendorst)
- 译 李博杰
- 责任编辑 刘美英
- 责任印制 胡 南
- ◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号
- 邮编 100164 电子邮件 315@ptpress.com.cn
- 网址 <https://www.ptpress.com.cn>
- 北京 印刷
- ◆ 开本: 800 × 1000 1/16
- 印张: 23.75 2025 年 5 月第 1 版
- 字数: 548 千字 2025 年 5 月北京第 1 次印刷
- 著作权合同登记号 图字: 01-2024-5494 号
-

定价: 159.80 元

读者服务热线: (010)84084456-6009 印装质量热线: (010)81055316

反盗版热线: (010)81055315

版权声明

Copyright © 2024 Jay Alammam and Maarten Pieter Grootendorst. All rights reserved.

Simplified Chinese edition, jointly published by O'Reilly Media, Inc. and Posts & Telecom Press, 2025. Authorized translation of the English edition, 2024 O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

All rights reserved including the rights of reproduction in whole or in part in any form.

英文原版由 O'Reilly Media, Inc. 出版，2024。

简体中文版由人民邮电出版社有限公司出版，2025。英文原版的翻译得到 O'Reilly Media, Inc. 的授权。此简体中文版的出版和销售得到出版权和销售权的所有者——O'Reilly Media, Inc. 的许可。

版权所有，未得书面许可，本书的任何部分和全部不得以任何形式重制。

目录

对本书的赞誉	xi
对本书中文版的赞誉	xiii
译者序	xv
中文版序	xxi
前言	xxiii

第一部分 理解语言模型

第 1 章 大语言模型简介	3
1.1 什么是语言人工智能	4
1.2 语言人工智能的近期发展史	4
1.2.1 将语言表示为词袋模型	5
1.2.2 用稠密向量嵌入获得更好的表示	7
1.2.3 嵌入的类型	9
1.2.4 使用注意力机制编解码上下文	10
1.2.5 “Attention Is All You Need”	13
1.2.6 表示模型：仅编码器模型	16
1.2.7 生成模型：仅解码器模型	18
1.2.8 生成式 AI 元年	20
1.3 “LLM”定义的演变	22
1.4 LLM 的训练范式	22
1.5 LLM 的应用	23
1.6 开发和使用负责任的 LLM	24
1.7 有限的资源就够了	25
1.8 与 LLM 交互	25

1.8.1 专有模型	26
1.8.2 开源模型	26
1.8.3 开源框架	27
1.9 生成你的第一段文本	28
1.10 小结	30
第 2 章 词元和嵌入	31
2.1 LLM 的分词	32
2.1.1 分词器如何处理语言模型的输入	32
2.1.2 下载和运行 LLM	33
2.1.3 分词器如何分解文本	36
2.1.4 词级、子词级、字符级与字节级分词	37
2.1.5 比较训练好的 LLM 分词器	39
2.1.6 分词器属性	47
2.2 词元嵌入	48
2.2.1 语言模型为其分词器的词表保存嵌入	49
2.2.2 使用语言模型创建与上下文相关的词嵌入	49
2.3 文本嵌入（用于句子和整篇文档）	52
2.4 LLM 之外的词嵌入	53
2.4.1 使用预训练词嵌入	53
2.4.2 word2vec 算法与对比训练	54
2.5 推荐系统中的嵌入	57
2.5.1 基于嵌入的歌曲推荐	57
2.5.2 训练歌曲嵌入模型	58
2.6 小结	60
第 3 章 LLM 的内部机制	61
3.1 Transformer 模型概述	62
3.1.1 已训练 Transformer LLM 的输入和输出	62
3.1.2 前向传播的组成	64
3.1.3 从概率分布中选择单个词元（采样 / 解码）	66
3.1.4 并行词元处理和上下文长度	68
3.1.5 通过缓存键 - 值加速生成过程	70
3.1.6 Transformer 块的内部结构	71
3.2 Transformer 架构的最新改进	79
3.2.1 更高效的注意力机制	79
3.2.2 Transformer 块	83
3.2.3 位置嵌入：RoPE	85
3.2.4 其他架构实验和改进	87
3.3 小结	87

第二部分 使用预训练语言模型

第 4 章 文本分类	91
4.1 电影评论的情感分析	92
4.2 使用表示模型进行文本分类	93
4.3 模型选择	94
4.4 使用特定任务模型	96
4.5 利用嵌入向量的分类任务	99
4.5.1 监督分类	99
4.5.2 没有标注数据怎么办	102
4.6 使用生成模型进行文本分类	105
4.6.1 使用 T5	106
4.6.2 使用 ChatGPT 进行分类	110
4.7 小结	113
第 5 章 文本聚类 and 主题建模	114
5.1 ArXiv 文章：计算与语言	115
5.2 文本聚类的通用流程	116
5.2.1 嵌入文档	116
5.2.2 嵌入向量降维	117
5.2.3 对降维后的嵌入向量进行聚类	119
5.2.4 检查生成的簇	120
5.3 从文本聚类到主题建模	122
5.3.1 BERTopic：一个模块化主题建模框架	124
5.3.2 添加特殊的“乐高积木块”	131
5.3.3 文本生成的“乐高积木块”	135
5.4 小结	138
第 6 章 提示工程	140
6.1 使用文本生成模型	140
6.1.1 选择文本生成模型	140
6.1.2 加载文本生成模型	141
6.1.3 控制模型输出	143
6.2 提示工程简介	145
6.2.1 提示词的基本要素	145
6.2.2 基于指令的提示词	147
6.3 高级提示工程	149
6.3.1 提示词的潜在复杂性	149
6.3.2 上下文学习：提供示例	152
6.3.3 链式提示：分解问题	153

6.4	使用生成模型进行推理	155
6.4.1	思维链：先思考再回答	156
6.4.2	自洽性：采样输出	159
6.4.3	思维树：探索中间步骤	160
6.5	输出验证	161
6.5.1	提供示例	162
6.5.2	语法：约束采样	164
6.6	小结	167
第 7 章	高级文本生成技术与工具	168
7.1	模型输入/输出：基于 LangChain 加载量化模型	169
7.2	链：扩展 LLM 的能力	171
7.2.1	链式架构的关键节点：提示词模板	172
7.2.2	多提示词链式架构	174
7.3	记忆：构建 LLM 的对话回溯能力	177
7.3.1	对话缓冲区	178
7.3.2	窗口式对话缓冲区	180
7.3.3	对话摘要	181
7.4	智能体：构建 LLM 系统	185
7.4.1	智能体的核心机制：递进式推理	186
7.4.2	LangChain 中的 ReAct 实现	187
7.5	小结	190
第 8 章	语义搜索与 RAG	191
8.1	语义搜索与 RAG 技术全景	191
8.2	语言模型驱动的语义搜索实践	193
8.2.1	稠密检索	193
8.2.2	重排序	204
8.2.3	检索评估指标体系	207
8.3	RAG	211
8.3.1	从搜索到 RAG	212
8.3.2	示例：使用 LLM API 进行基于知识的生成	213
8.3.3	示例：使用本地模型的 RAG	213
8.3.4	高级 RAG 技术	215
8.3.5	RAG 效果评估	217
8.4	小结	218
第 9 章	多模态 LLM	219
9.1	视觉 Transformer	220
9.2	多模态嵌入模型	222
9.2.1	CLIP：构建跨模态桥梁	224

9.2.2	CLIP 的跨模态嵌入生成机制	224
9.2.3	OpenCLIP	226
9.3	让文本生成模型具备多模态能力	231
9.3.1	BLIP-2: 跨越模态鸿沟	231
9.3.2	多模态输入预处理	235
9.3.3	用例 1: 图像描述	237
9.3.4	用例 2: 基于聊天的多模态提示词	240
9.4	小结	242

第三部分 训练和微调语言模型

第 10 章	构建文本嵌入模型	247
10.1	嵌入模型	247
10.2	什么是对比学习	249
10.3	SBERT	251
10.4	构建嵌入模型	253
10.4.1	生成对比样本	253
10.4.2	训练模型	254
10.4.3	深入评估	257
10.4.4	损失函数	258
10.5	微调嵌入模型	265
10.5.1	监督学习	265
10.5.2	增强型 SBERT	267
10.6	无监督学习	271
10.6.1	TSDAE	272
10.6.2	使用 TSDAE 进行领域适配	275
10.7	小结	276
第 11 章	为分类任务微调表示模型	277
11.1	监督分类	277
11.1.1	微调预训练的 BERT 模型	279
11.1.2	冻结层	281
11.2	少样本分类	286
11.2.1	SetFit: 少样本场景下的高效微调方案	286
11.2.2	少样本分类的微调	290
11.3	基于掩码语言建模的继续预训练	292
11.4	命名实体识别	297
11.4.1	数据准备	298
11.4.2	命名实体识别的微调	303
11.5	小结	305

第 12 章 微调生成模型	306
12.1 LLM 训练三步走：预训练、监督微调和偏好调优	306
12.2 监督微调	308
12.2.1 全量微调	308
12.2.2 参数高效微调	309
12.3 使用 QLoRA 进行指令微调	317
12.3.1 模板化指令数据	317
12.3.2 模型量化	318
12.3.3 LoRA 配置	319
12.3.4 训练配置	320
12.3.5 训练	321
12.3.6 合并权重	322
12.4 评估生成模型	322
12.4.1 词级指标	323
12.4.2 基准测试	323
12.4.3 排行榜	324
12.4.4 自动评估	325
12.4.5 人工评估	325
12.5 偏好调优、对齐	326
12.6 使用奖励模型实现偏好评估自动化	327
12.6.1 奖励模型的输入和输出	328
12.6.2 训练奖励模型	329
12.6.3 训练无奖励模型	332
12.7 使用 DPO 进行偏好调优	333
12.7.1 对齐数据的模板化	333
12.7.2 模型量化	334
12.7.3 训练配置	335
12.7.4 训练	336
12.8 小结	337
附录 图解 DeepSeek-R1	338
后记	349

LLM的内部机制

在了解了分词和词嵌入之后，我们已经准备好深入探究语言模型的工作原理了。在本章中，我们将探讨 Transformer LLM 的主要工作原理。我们将重点关注文本生成模型，以便更深入地理解生成式 LLM。

我们将探讨相关概念并通过一些代码示例来演示这些概念。我们先加载一个语言模型，并定义流水线，以备生成文本。在第一次阅读时，你可以跳过代码部分，专注于理解概念。然后在第二次阅读时，通过代码来动手应用这些概念。

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

# 加载模型和分词器
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")

model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)

# 创建流水线
generator = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    return_full_text=False,
    max_new_tokens=50,
    do_sample=False,
)
```

3.1 Transformer模型概述

我们先了解 Transformer 的基础知识，再了解 Transformer 模型自 2017 年问世以来的改进。

3.1.1 已训练Transformer LLM的输入和输出

要理解 Transformer LLM 的行为，最常见的方式是将其视为一个接收文本输入并生成响应文本的软件系统。当一个足够大的文本输入 - 输出模型在足够大的高质量数据集上完成训练后，它能够生成令人印象深刻且实用的输出。图 3-1 展示了一个用于撰写电子邮件的此类模型。

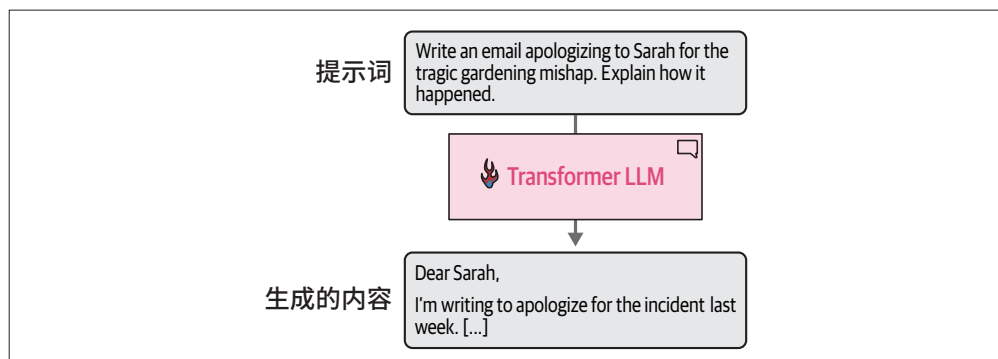


图 3-1：从整体来看，Transformer LLM 接收提示词输入，并输出生成的文本

模型并不是一次性生成所有文本，而是一次生成一个词元。图 3-2 展示了响应输入提示词时的四个词元生成步骤。每个词元生成步骤都是模型的一次前向传播（在机器学习中，前向传播指的是输入进入神经网络并流经计算图，最终在另一端产生输出所需的计算过程）。

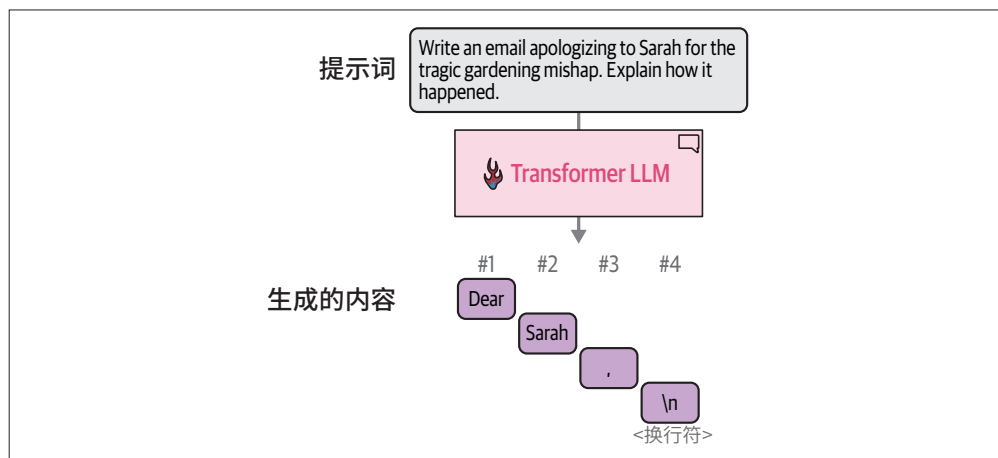


图 3-2：Transformer LLM 一次生成一个词元，而不是一次生成全部文本

在生成当前词元后，我们将输出词元追加到输入提示词的末尾，从而调整下一次生成的输入提示词。我们可以在图 3-3 中看到这一点。

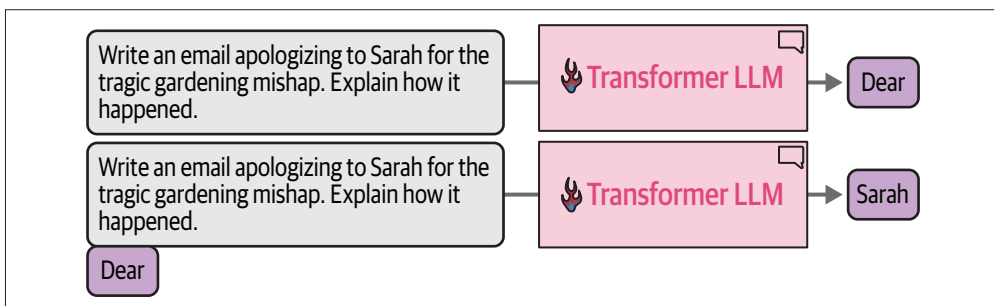


图 3-3: 输出词元被追加到提示词末尾后，得到的新文本会再次提供给模型进行另一次前向传播，以生成下一个词元

这让我们对模型有了更准确的认识，它只是基于输入提示词预测下一个词元。神经网络外围的软件基本上就是在循环运行这个模型，按顺序扩展生成的文本，直到完成。

在机器学习中，有一个专门的词用来描述使用早期预测来进行后续预测的模型（例如，模型使用生成的第一个词元来生成第二个词元），这类模型被称为**自回归模型**（autoregressive model）。这就是为什么文本生成式 LLM 也被称为自回归模型。这一名称通常用于区分文本生成模型与像 BERT 这样的非自回归的文本表示模型。

如下所示，当我们使用 LLM 生成文本时，在底层发生的就是这种自回归的、逐词元生成的过程。

```
prompt = "Write an email apologizing to Sarah for the tragic gardening mishap.  
Explain how it happened."  
  
output = generator(prompt)  
  
print(output[0]['generated_text'])
```

这生成了以下文本：

```
Solution 1:  
Subject: My Sincere Apologies for the Gardening Mishap  
Dear Sarah,  
I hope this message finds you well. I am writing to express my deep
```

我们可以看到模型从 Subject（主题）一行开始写这封邮件。它突然停止，是因为达到了我们设置的 `max_new_tokens` 为 50 的限制。如果我们将这个值设置得更大，它会继续生成内容，直到完成这封邮件。

3.1.2 前向传播的组成

除了循环之外，前向传播还有两个关键的内部组件：分词器和语言建模头（language modeling head, LM head）。图 3-4 展示了这些组件在系统中的位置。在上一章中，我们看到了分词器如何将文本分解成词元 ID 序列，然后用作模型的输入。

分词器之后是神经网络，由一系列 Transformer 块堆叠而成，负责执行所有的处理工作。在这些堆叠的块之后是语言建模头，它将 Transformer 块的输出转换为预测下一个词元的概率分数。

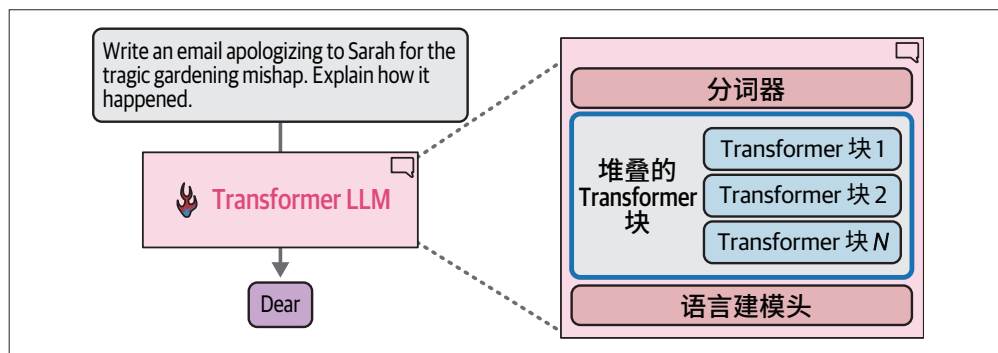


图 3-4：Transformer LLM 由分词器、堆叠的 Transformer 块和语言建模头组成

回顾第 2 章的内容，分词器包含一个词元表，即分词器的词表。模型为词表中的每个词元都关联了一个向量表示（词元嵌入）。图 3-5 展示了一个拥有 50 000 个词元的词表及其对应的词元嵌入。

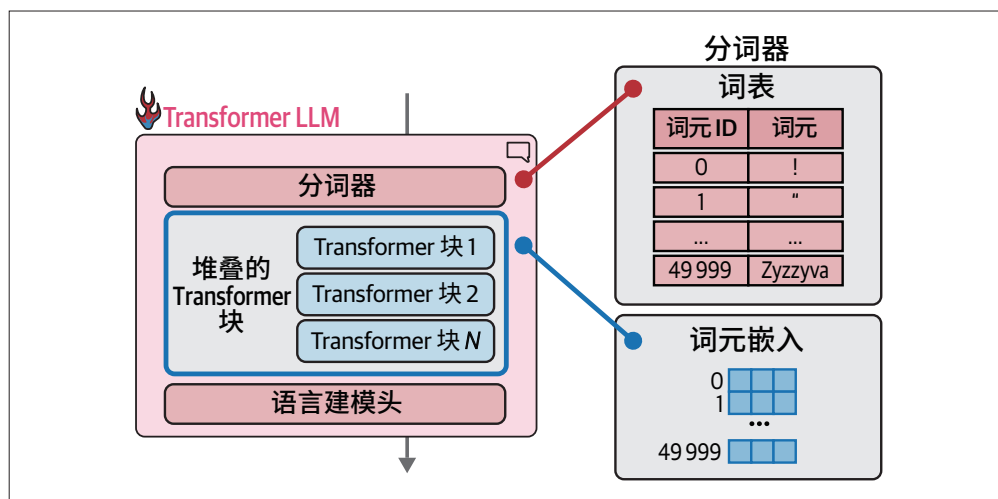


图 3-5：分词器拥有 50 000 个词元的词表，模型为这些词元关联了词元嵌入

计算流按照箭头方向从上到下进行。对于每个生成的词元，处理过程会按顺序依次经过堆叠成一列的所有 Transformer 块，然后到达语言建模头，最后输出下一个词元的概率分布，如图 3-6 所示。

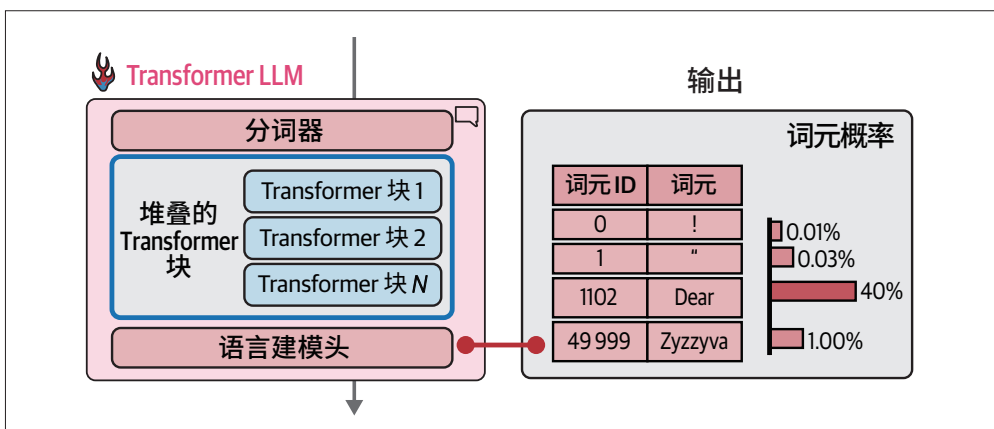


图 3-6: 在前向传播结束时，模型为词表中的每个词元预测一个概率分数

语言建模头本身是一个简单的神经网络层。它可以连接到堆叠的 Transformer 块上的多种可能的“头”之一，用于构建不同类型的系统。其他类型的 Transformer 头包括序列分类头和词元分类头。

我们只需打印模型变量，就可以按顺序显示所有层。对于这个模型，我们得到：

```
Phi3ForCausalLM(
  (model): Phi3Model(
    (embed_tokens): Embedding(32064, 3072, padding_idx=32000)
    (embed_dropout): Dropout(p=0.0, inplace=False)
    (layers): ModuleList(
      (0-31): 32 x Phi3DecoderLayer(
        (self_attn): Phi3Attention(
          (o_proj): Linear(in_features=3072, out_features=3072, bias=False)
          (qkv_proj): Linear(in_features=3072, out_features=9216, bias=False)
          (rotary_emb): Phi3RotaryEmbedding()
        )
        (mlp): Phi3MLP(
          (gate_up_proj): Linear(in_features=3072, out_features=16384, bias=False)
          (down_proj): Linear(in_features=8192, out_features=3072, bias=False)
          (activation_fn): SiLU()
        )
        (input_layernorm): Phi3RMSNorm()
        (resid_attn_dropout): Dropout(p=0.0, inplace=False)
        (resid_mlp_dropout): Dropout(p=0.0, inplace=False)
        (post_attention_layernorm): Phi3RMSNorm()
      )
    )
  )
)
```

```

    )
    (norm): Phi3RMSNorm()
  )
  (lm_head): Linear(in_features=3072, out_features=32064, bias=False)
)

```

观察这个结构，我们可以注意到以下几个重点。

- 这个结构展示了模型的各种嵌套层。模型的主要部分标记为 `model`，随后是 `lm_head`。
- 在 `Phi3Model` 内部，我们可以看到嵌入矩阵 `embed_tokens` 及其维度。它有 32 064 个词元，每个词元的向量大小为 3072。
- 暂时跳过 `dropout` 层，我们可以看到下一个主要组件是堆叠的 Transformer 解码器层。它包含 32 个 `Phi3DecoderLayer` 类型的块。
- 这些 Transformer 块中的每一个都包含一个注意力层和一个前馈神经网络（也称为 MLP 或多层感知器）。我们将在本章后面详细介绍这些内容。
- 最后，我们看到 `lm_head` 接收一个大小为 3072 的向量，并输出一个大小等于模型所知词元数量的向量。该输出是每个词元的概率分数，帮助我们选择输出词元。

3.1.3 从概率分布中选择单个词元（采样/解码）

在处理结束时，模型会为词表中的每个词元输出一个概率分数，正如我们在图 3-6 中看到的那样。从概率分布中选择单个词元的方法称为**解码策略**。图 3-7 展示了示例是如何选择词元 `Dear` 的。

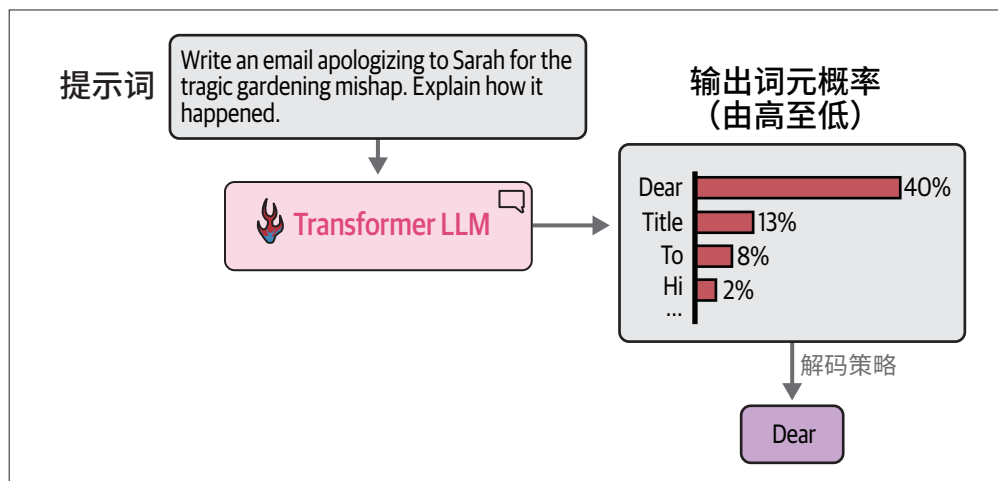


图 3-7：经过模型前向传播后，基于上下文，模型可能输出的概率最高的几个词元。我们的解码策略通过基于概率的采样来决定输出哪个词元

最简单的解码策略就是始终选择概率分数最高的词元。但在实践中，对于大多数使用场景来说，这种方法往往无法产生最佳输出。一个更好的方法是引入一些随机性，有时选择概率第二高或第三高的词元。用统计学家的话来说，这种思想就是根据概率分数对概率分布进行采样。

对于图 3-7 中的例子来说，如果 Dear 作为下一个词元的概率为 40%，那么它被选中的概率就是 40%（而不是像贪心搜索那样，直接选择这个得分最高的词元）。这样，其他词元也有机会根据其分数被选中。

每次都选择概率分数最高的词元的策略被称为贪心解码。这就是在 LLM 中将温度 (temperature) 参数设为零时会发生的情况。我们将在第 6 章讨论温度的概念。

让我们仔细看看演示这个过程的代码。在这个代码块中，我们将输入词元传递给模型，然后传给 `lm_head`：

```
prompt = "The capital of France is"

# 对输入提示词进行分词
input_ids = tokenizer(prompt, return_tensors="pt").input_ids

# 将词元ID移动到GPU上
input_ids = input_ids.to("cuda")

# 获取模型在lm_head之前的输出
model_output = model.model(input_ids)

# 获取lm_head的输出
lm_head_output = model.lm_head(model_output[0])
```

现在，`lm_head_output` 的形状是 `[1, 5, 32064]`。我们可以使用 `lm_head_output[0,-1]` 来访问最后生成的词元的概率分数，其中索引 0 用于批次维度，表示一批数据中的第一个，索引 -1 用于获取序列中的最后一个词元。现在我们得到了全部 32 064 个词元的概率分数列表。接下来我们可以获取得分最高的词元 ID，然后解码，以得到生成的输出词元的文本：

```
token_id = lm_head_output[0,-1].argmax(-1)
tokenizer.decode(token_id)
```

结果是：

```
Paris
```

3.1.4 并行词元处理和上下文长度

Transformer 最引人注目的特性之一是，它比之前的语言处理神经网络架构更适合并行计算。在文本生成中，观察每个词元是如何处理的，就能初步了解这一点。我们从上一章知道，分词器会将文本分解成词元。然后每个输入词元都会流经自己的计算路径（这种直觉有助于我们更轻松地理解）。我们可以在图 3-8 中看到这些独立的计算流（也可以理解为处理路径）。

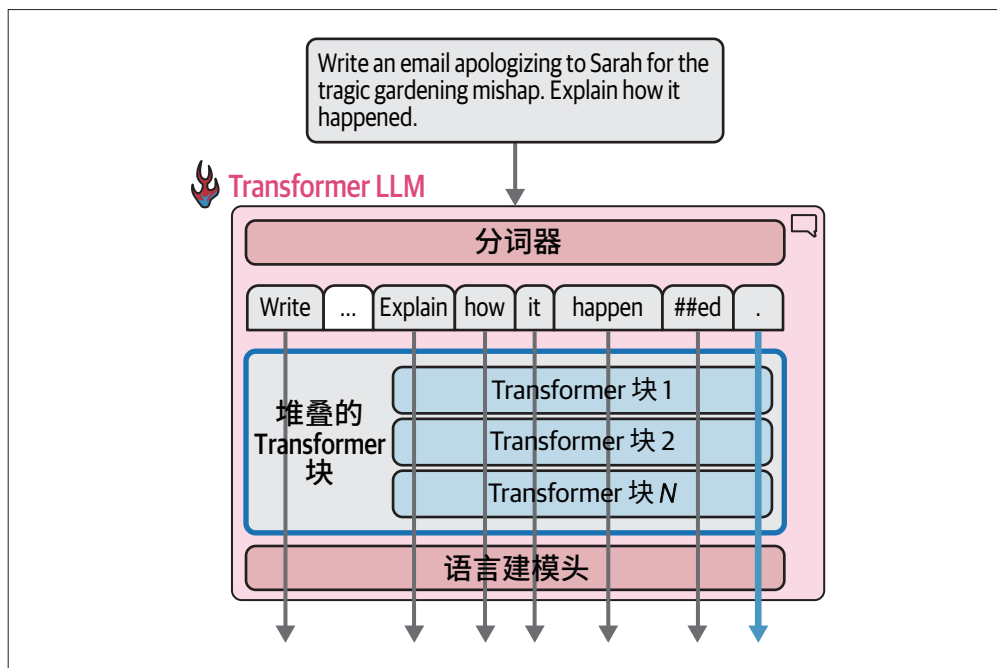


图 3-8: 每个词元都通过自己的计算流进行处理(之后我们会看到,它们在注意力步骤中会有一些交互)

当前的 Transformer 模型对一次可以处理的词元数量有限制,这个限制被称为模型的上下文长度。一个具有 4K 上下文长度的模型只能处理 4000 个词元,也就是只有 4000 条这样的流。

每条计算流都从一个输入向量开始(包括嵌入向量和一些位置信息。我们将在本章后面讨论位置嵌入)。在流的末尾,另一个向量作为模型处理的结果出现,如图 3-9 所示。

Transformer LLM

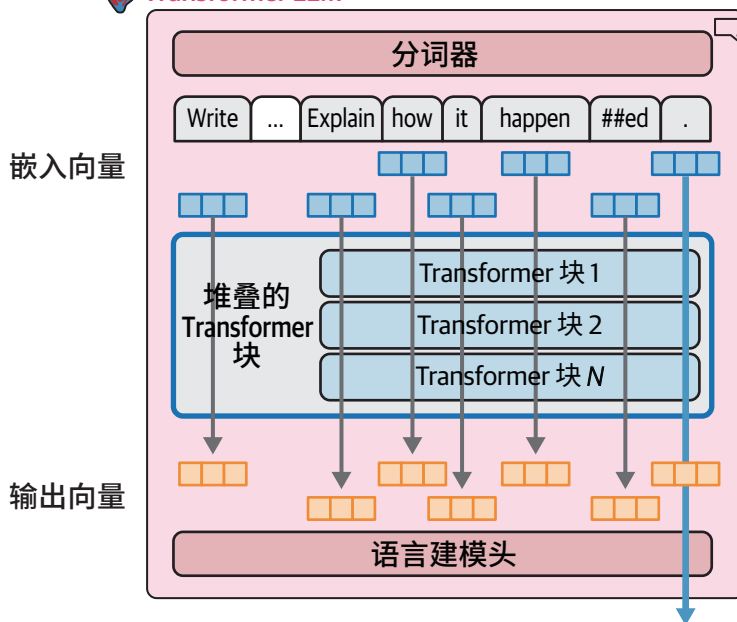


图 3-9: 每条处理流接收一个向量作为输入, 并生成一个大小相同的最终结果向量 (这一大小通常称为模型维度)

对于文本生成来说, 只有最后一条计算流的输出结果用于预测下一个词元。当语言建模头计算下一个词元的概率时, 该输出向量是唯一的输入。

你可能会疑惑, 既然最终只用到最后一个词元的输出, 为什么还需要所有的计算流? 答案是, 之前的流的计算结果是最终的流所必需的。没错, 我们不会使用它们的最终输出向量, 但会在每个 Transformer 块的注意力机制中使用其早期输出。

如果你在跟着代码示例学习, 回想一下, `lm_head` 的输出形式为 `[1, 5, 32064]`, 这是因为它的输入形式为 `[1, 5, 3072]`, 代表一个批次中包含一个输入字符串, 该字符串包含 5 个词元, 每个词元都由一个大小为 3072 的向量表示, 这些向量对应着堆叠的 Transformer 块处理后的输出向量。

我们可以打印这些矩阵, 以查看它们的维度:

```
model_output[0].shape
```

输出结果为:

```
torch.Size([1, 5, 3072])
```

同样，我们可以打印语言建模头的输出：

```
lm_head_output.shape
```

输出结果为：

```
torch.Size([1, 5, 32064])
```

3.1.5 通过缓存键-值加速生成过程

回想一下，在生成第二个词元时，我们只是简单地将输出词元追加到输入的末尾，然后再次通过模型进行前向传播。如果模型能够缓存之前的计算结果（特别是注意力机制中的一些特定向量），就不需要重复计算之前的流，而只需要计算最后一条流了。这种优化技术被称为键-值（key-value, KV）缓存，它能显著加快生成过程。键和值是注意力机制的核心组件，我们将在本章后面详细介绍。

如图 3-10 所示，在生成第二个词元时，由于我们缓存了之前流的结果，只有一条计算流是活跃的。

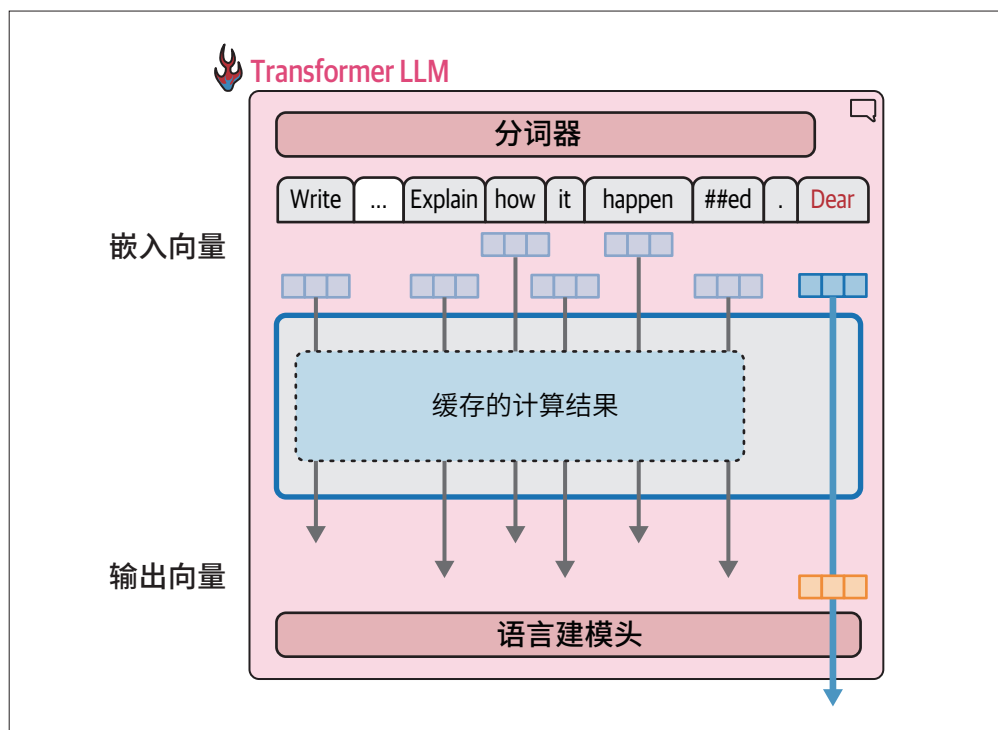


图 3-10：在生成文本时，重要的是缓存之前词元的计算结果，而不是反复进行相同的计算

在 Hugging Face Transformers 中，缓存默认是启用的，可以将 `use_cache` 设置为 `False` 来禁用。我们可以请求一个较长的生成任务，并对比启用和禁用缓存的生成时间，从而感受差异：

```
prompt = "Write a very long email apologizing to Sarah for the tragic gardening mishap. Explain how it happened."
# 对输入提示词进行分词
input_ids = tokenizer(prompt, return_tensors="pt").input_ids
input_ids = input_ids.to("cuda")
```

然后我们测量启用缓存后生成 100 个词元所需的时间。我们可以在 Jupyter 或 Colab 中使用 `%%timeit` 魔法命令来计时（它会多次运行命令并取用时的平均值）：

```
%%timeit -n 1
# 生成文本
generation_output = model.generate(
    input_ids=input_ids,
    max_new_tokens=100,
    use_cache=True
)
```

在配备 T4 GPU 的 Colab 上，这需要 4.5 秒。如果我们禁用缓存，需要多长时间呢？

```
%%timeit -n 1
# 生成文本
generation_output = model.generate(
    input_ids=input_ids,
    max_new_tokens=100,
    use_cache=False
)
```

用时变成了 21.8 秒，差异非常显著。事实上，从用户体验的角度来看，即使是 4 秒的生成时间，对于正盯着屏幕等待模型输出的用户来说也是很长的。这就是为什么 LLM API 会在模型生成过程中流式输出词元，而不是等待整个生成过程完成再输出。

3.1.6 Transformer 块的内部结构

现在我们来讨论 Transformer 模型的核心处理单元——Transformer 块。如图 3-11 所示，Transformer LLM 由一系列 Transformer 块组成（在原始 Transformer 论文中约为 6 个，而在许多 LLM 中超过 100 个）。每个块处理其输入，然后将其处理结果传递给下一个块。

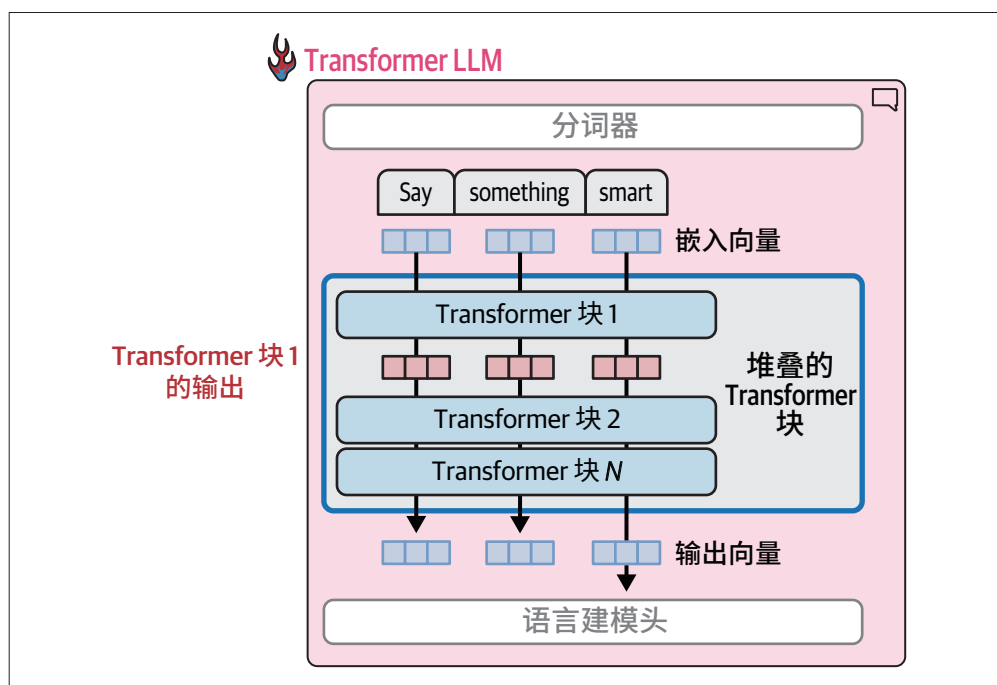


图 3-11: Transformer LLM 的大部分处理过程发生在一系列 Transformer 块中，每个块将其处理结果作为输入传递给下一个块

Transformer 块由以下两个首尾相接的组件构成（图 3-12）。

- 自注意力层，主要负责整合来自其他输入词元和位置的相关信息。
- 前馈神经网络层，包含模型的主要处理能力。

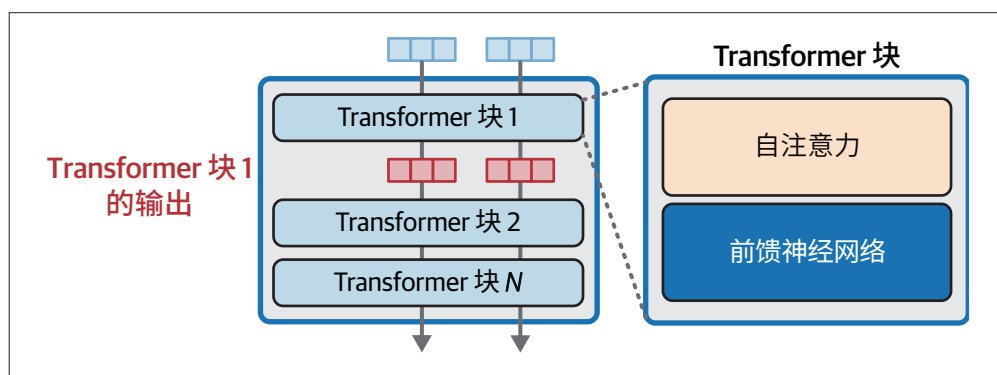


图 3-12: Transformer 块由一个自注意力层和一个前馈神经网络层组成

1. 前馈神经网络层概览

让我们用一个简单的例子来理解前馈神经网络的工作原理：我们向语言模型输入“The Shawshank”，期望它生成“Redemption”（指 1994 年的电影《肖申克的救赎》）。

如图 3-13 所示，前馈神经网络（分布在所有模型层中）就是这些信息的来源。当模型在大规模文本数据（包含大量对“The Shawshank Redemption”的引用）上完成训练后，它学习并存储了完成这项任务所需的信息（和行为）。

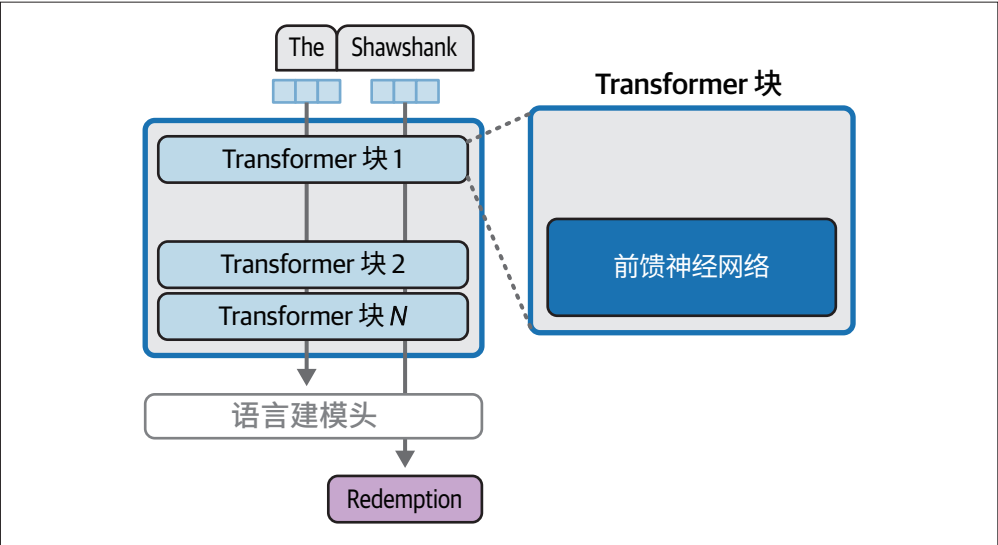


图 3-13: Transformer 块中的前馈神经网络层可能承担了模型大部分的记忆和插值工作

要想成功训练一个 LLM，需要让它记住大量信息。但它并不仅仅是一个大型数据库。记忆只是生成出色文本的众多要素之一。模型能够利用相同的机制在数据点之间进行插值，识别更复杂的模式，从而实现泛化，这意味着它能够很好地处理以前从未见过、不在训练数据集中的输入。



当你使用现代商用 LLM 时，你得到的输出并不是前文中严格意义上的“语言模型”的输出。如果向 GPT-4 这样的对话型 LLM 输入“The Shawshank”，它会输出：

"The Shawshank Redemption" is a 1994 film directed by Frank Darabont and is based on the novella "Rita Hayworth and Shawshank Redemption" written by Stephen King. ...etc.

这是因为原始语言模型（如 GPT-3）对用户来说很难用，因此语言模型需要通过指令微调和基于人类偏好与反馈的微调，来满足人们对模型输出的期望。

2. 自注意力层概览

上下文对于正确建模语言至关重要。仅仅依靠基于前一个词元的简单记忆和插值是远远不够的。我们能得出这个结论，是因为这是神经网络出现之前构建语言模型的主要方法之一（参见 Daniel Jurafsky 和 James H. Martin 的 *Speech and Language Processing* 第 3 章“*N*-gram Language Models”）。

注意力机制帮助模型在处理特定词元时整合上下文信息。考虑以下提示词：

The dog chased the squirrel because it （狗追松鼠，因为它）

为了预测 it 之后的内容，模型需要知道 it 指代什么，是狗还是松鼠？

在已训练的 Transformer LLM 中，注意力机制负责做出这种判断。注意力机制将上下文信息添加到 it 词元的表示中。我们可以在图 3-14 中看到一个简单的示例。

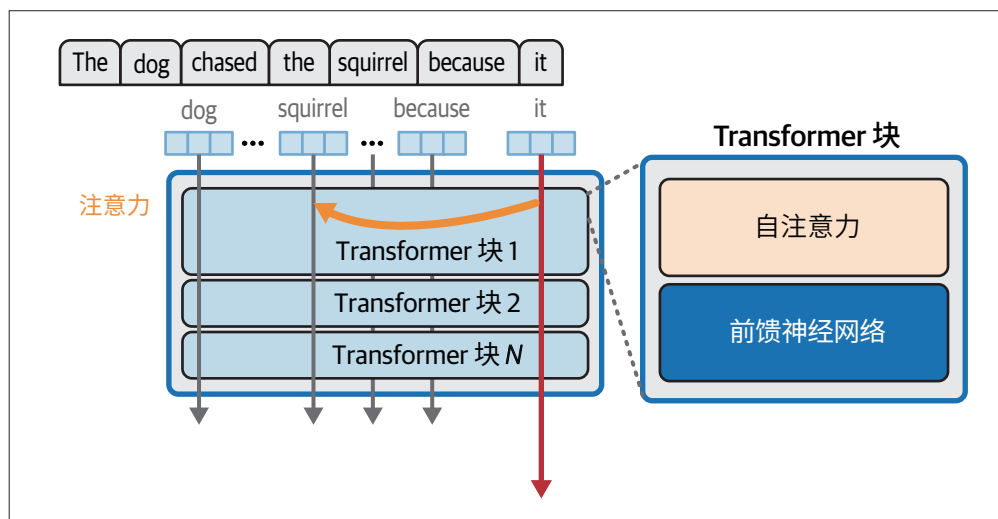


图 3-14：自注意力层整合了来自前序位置的相关信息，用于处理当前词元

模型是基于从训练数据集中观察和学习到的模式来实现这一点的。前文中的句子可能提供了更多线索，比如前文中用 she 指代狗，那就表明 it 指代的是松鼠。

3. 注意力机制的重要性

我们有必要深入地了解注意力机制。图 3-15 展示了注意力机制的极简版本。它显示了多个进入自注意力层的词元位置，最后一个是当前正在处理的位置（粉色箭头）。注意力机制作用于该位置的输入向量，它将上下文中的相关信息整合到该位置的输出向量中。

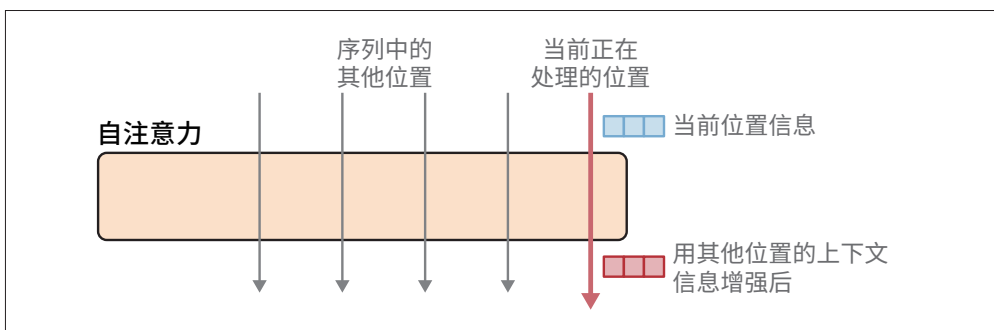


图 3-15: 注意力机制的简化示意图：一个输入序列和当前正在处理的位置。由于我们主要关注的是这个位置，图中显示了一个输入向量和一个输出向量，其中输出向量基于注意力机制整合了序列中前面元素的信息

注意力机制包含以下两个主要步骤。

- 对当前处理的词元（粉色箭头所示）与之前输入词元的相关性评分。
- 利用这些分数，将不同位置的信息组合成单一的输出向量。

图 3-16 展示了这两个步骤。

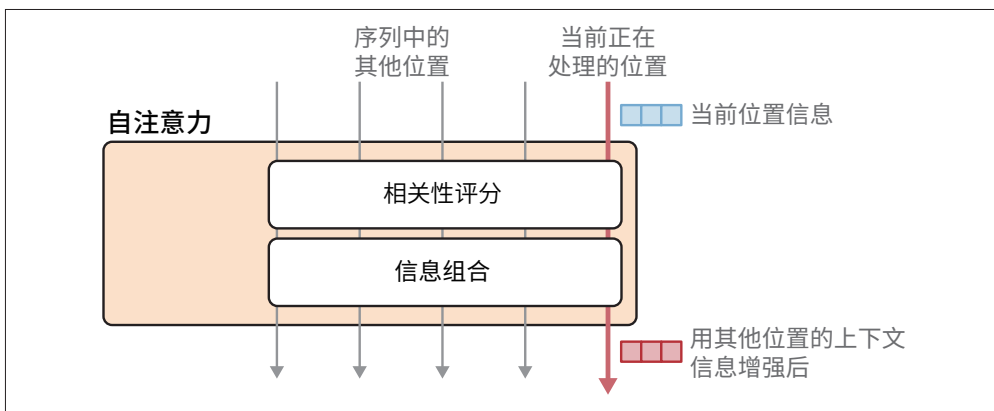


图 3-16: 注意力机制由两个主要步骤组成：对每个位置进行相关性评分，然后基于这些评分进行信息组合

为了赋予 Transformer 更强大的注意力能力，注意力机制被复制多份，并行执行。这些并行的注意力执行过程被称为**注意力头**（attention head）。这提高了模型对输入序列中复杂模式的建模能力，使其能够同时关注不同的模式。

图 3-17 直观地展示了多个注意力头如何并行运行，包括前面的信息分割步骤和后面的所有注意力头的结果合并步骤。

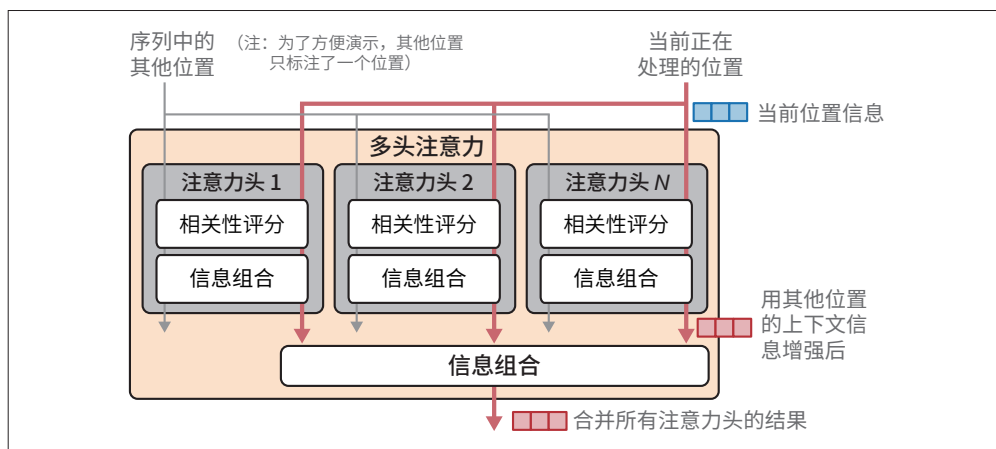


图 3-17: 通过并行执行多次注意力计算来获得更好的 LLM，提高模型关注不同类型的信息的能力

4. 注意力的计算方式

让我们来看看单个注意力头内的注意力是如何计算的。在开始计算之前，我们先观察以下起始状态。

- (生成式 LLM 中的) 注意力层正在为单个位置处理注意力。
- 该层的输入包括：
 - 当前位置或词元的向量表示
 - 前序词元的向量表示
- 目标是为当前位置生成一个新的表示，其中包含来自前序词元的相关信息。例如，我们正在处理句子“Sarah fed the cat because it” (Sarah 喂了猫，因为它) 的最后一个位置，我们希望 it 表示那只猫，所以注意力机制会融入来自 cat 词元的“猫的信息”。
- 训练过程会产生三个投影矩阵，用于生成参与计算的组件：
 - 查询投影矩阵
 - 键投影矩阵
 - 值投影矩阵

图 3-18 展示了注意力计算开始前这些组件的起始状态。为简单起见，我们只考察一个注意力头，因为其他注意力头进行的是同样的计算，只是使用各自的投影矩阵。

注意力首先将输入与投影矩阵相乘，得到三个新矩阵，称为查询矩阵、键矩阵和值矩阵。这些矩阵包含了投影到三个不同空间的输入词元信息，用于执行注意力的两个步骤：

- 相关性评分
- 信息组合

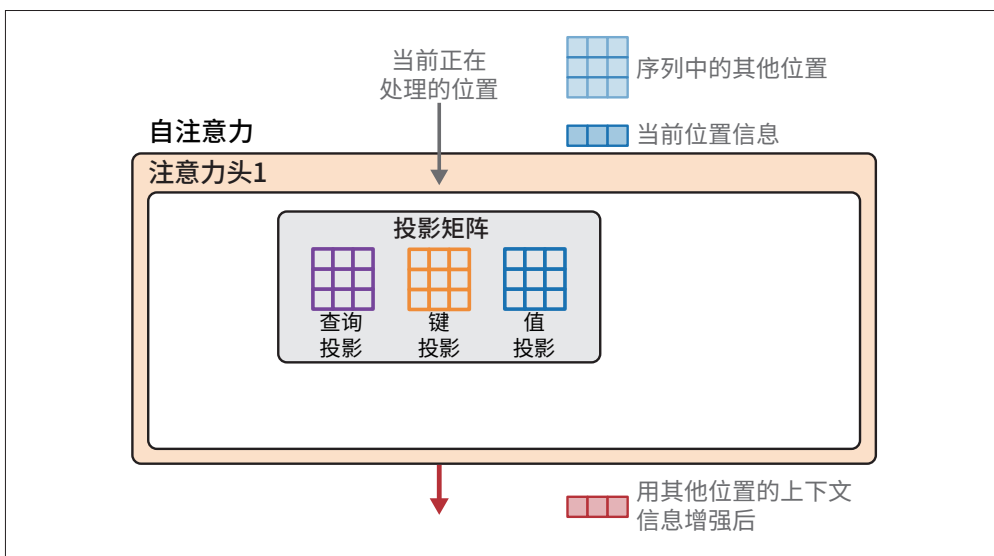


图 3-18：在开始注意力计算之前，该层的输入和查询、键、值的投影矩阵已准备就绪

图 3-19 展示了这三个新矩阵，以及三个矩阵的最后一行如何与当前位置相关联，而上面的行则与前序位置相关联。

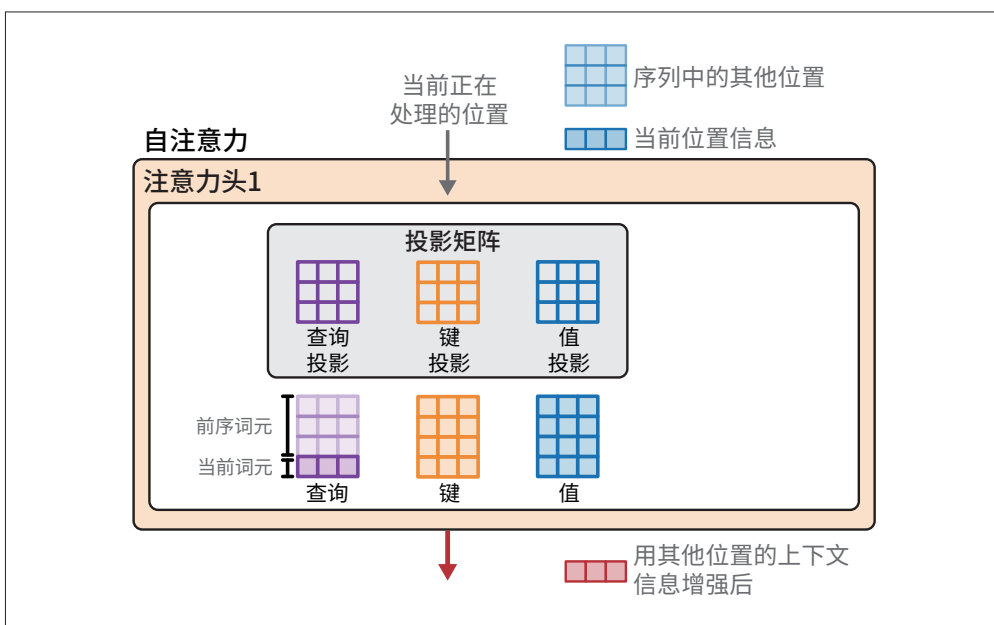


图 3-19：注意力是通过查询矩阵、键矩阵和值矩阵的交互来执行的。这些矩阵是将层的输入与投影矩阵相乘得到的

5. 自注意力：相关性评分

在生成式 Transformer 中，一次生成一个词元意味着一次处理一个位置。因此，注意力机制在这里只关注这一个位置（当前位置），以及如何从其他位置提取信息来为当前位置提供参考。

注意力机制的相关性评分步骤是通过将当前位置的查询向量与键矩阵相乘来实现的。这一操作会产生一组分数，用以衡量当前位置之前的每一个词元的相关性。接下来，通过 softmax 操作对这些分数进行归一化，使它们的和为 1。图 3-20 展示了计算得到的相关性分数。

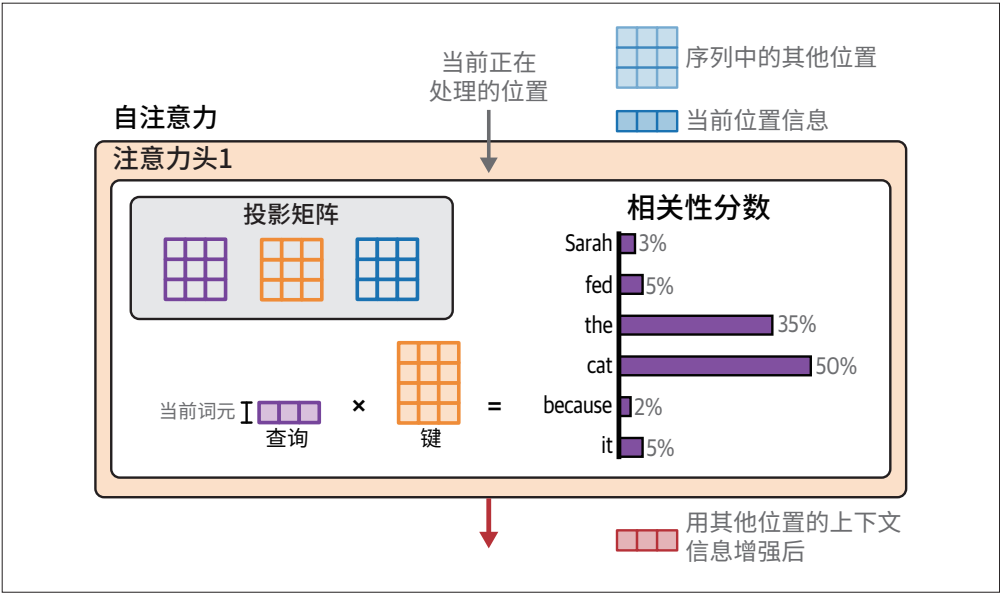


图 3-20：将与当前位置相关的查询向量和键矩阵相乘，以完成对前序词元的相关性评分

6. 自注意力：信息组合

有了相关性分数，我们用每个词元对应的值向量乘以该词元的分数，然后将这些结果向量相加，就得到了注意力步骤的输出，如图 3-21 所示。